moOngy.
Minds on the move

React Avançado · Módulo 2

Sessão 3

Rendering e composição

ter 10 nov 2026 · 19h–21h

João Gonçalves · joaogoncalves.net

Recap da Sessão 2

Parámos no FlightSearch — tag sessao-02

Lista e cesto

- `FlightList`: `key={flight.id}`
- O cesto vive no **pai**: `Record<number, boolean>`
- `Badge` estende o `<span>`

React 19

- `ref` como prop — adeus `forwardRef`
- `useId` liga `label` e `input`
- `<title>` em qualquer componente

FlightSearch

- State + **derivados** (`canSearch`, `selectedCount`)
- Setters funcionais
- Clear → `fromRef.current?.focus()`

Por resolver: escrever no From faz renderizar **todos** os `FlightCard`. Porquê — e como evitar **sem** `memo`? → hoje.

Agenda de hoje

✓	**Exercício da Sessão 2**	a ideia principal · resolução no repo no fim da sessão	≈ 1 min
01	**Revisão de Hooks**	snapshot · derivado sem effect · regras dos Hooks	≈ 10 min
02	**Rendering e reconciliation**	porque renderiza · render → commit · keys	≈ 30 min
03	**Composição**	moving state down · children · slots · AppShell	≈ 45 min

Memoization e React Compiler passam para a **Sessão 4** — primeiro compor, depois memoizar.

Exercício da Sessão 2 · a ideia principal

Resolução completa no repo: curso/exercicios/sessao-02-resolucao.md (no fim desta sessão)

FlightSearch · a lista visível é derivada

```
const max = Number(filter.maxPrice);   // '' → 0 → sem limite
const filtered = flights
  .filter((f) => !onlyOnTime || !f.delayed)
  .filter((f) => !max || lowestPrice(f) <= max);

const visibleFlights = sort === 'none'
  ? filtered
  : filtered.toSorted((a, b) => …);    // não muta

setSort(event.target.value as SortOrder);
```

A reter

- Lista **derivada**: sem state nem effect
- Checkbox → `event.target.checked`
- `as SortOrder` é seguro: as `<option>` são nossas
- `SearchInput` reutilizado pelo `...rest`

Ao ordenar, a seleção não se baralhou. Porquê? A `key={flight.id}` — vemos daqui a pouco.

Bloco 01

Revisão de Hooks

Três perguntas para afinar o modelo mental



Pergunta 1: quanto vale count depois de um clique?

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}
```

Pensem antes de responder

- Três vezes `setCount(count + 1)`
- Depois de **um** clique: `count` vale…?
- 1, 2 ou 3?

Pergunta 1: quanto vale count depois de um clique?

```
const [count, setCount] = useState(0);

function handleClick() {
  setCount(count + 1);
  setCount(count + 1);
  setCount(count + 1);
}
```

Resposta

- **1** — em cada render, `count` é um snapshot
- As três chamadas são `setCount(0 + 1)`
- `setCount(c => c + 1)` × 3 → **3**

Quando o novo valor depende do anterior: **forma funcional**. Foi o que fizemos no `setBasket`.

Pergunta 2: este useEffect é necessário?

```
const [filter, setFilter] = useState(empty);
const [canSearch, setCanSearch] = useState(false);

useEffect(() => {
  setCanSearch(filter.from !== ''
               && filter.to !== '');
}, [filter]);
```

Pensem antes de responder

- O `canSearch` precisa de ser **state**?
- E este `useEffect`, é necessário?

Pergunta 2: este useEffect é necessário?

```
const [filter, setFilter] = useState(empty);
const [canSearch, setCanSearch] = useState(false);

useEffect(() => {
  setCanSearch(filter.from !== ''
               && filter.to !== '');
}, [filter]);
```

```
const [filter, setFilter] = useState(empty);

// derivado: calcula-se no render
const canSearch = filter.from !== ''
                  && filter.to !== '';
```

Resposta: não

- Derivado de state → calcula-se no render: um render em vez de dois, e nunca fica dessincronizado
- `useEffect` é para sincronizar com o **exterior**: rede, DOM, timers, subscriptions

Pergunta 3: o que está errado aqui?

```
function FlightCard({ flight }: Props) {
  if (!flight.delayed) {
    return <p>On time</p>;
  }
  const [open, setOpen] = useState(false);
  // …
}
```

Pensem antes de responder

- Compila? Funciona?
- O que pode correr mal quando `flight.delayed` muda?

Pergunta 3: o que está errado aqui?

```
function FlightCard({ flight }: Props) {
  if (!flight.delayed) {
    return <p>On time</p>;
  }
  const [open, setOpen] = useState(false);   // ▫
  // …
}
```

Resposta

- Hook depois de um `return` condicional
- O React identifica os Hooks pela **ordem** de chamada
- O oxlint avisa: `react/rules-of-hooks`

Hooks sempre no topo: nunca em `if`, loops ou depois de um `return`. (Exceção nova: `use()` — Sessão 5.)

Bloco 02

Rendering e reconciliation

Porque é que um componente volta a renderizar — e o que o React faz a seguir

Porque é que um componente volta a renderizar?

Três razões — e só três

1 · O seu state muda

- `setX(…)` com um valor novo
- É o único "gatilho" que escrevemos

2 · O pai renderiza

- Todos os filhos renderizam
- **Mesmo** com props iguais

3 · Um Context muda

- Quem o lê renderiza
- Context a sério: mais à frente

No Skyline: Select num card → `setBasket` no `FlightSearch` → renderizam o `FlightSearch`, a `FlightList` e **todos** os `FlightCard` — só um mudou.

Mito: "renderiza quando as props mudam". Não — renderiza porque **o pai renderizou**. As props só contam com `memo` (Sessão 4).

Renderizar não é mexer no DOM

Render → reconciliation → commit

Trigger
`setState` (ou o pai)

Render
corre a função, devolve JSX novo

Reconciliation
compara com o JSX anterior

Commit
só as diferenças vão para o DOM

Como o React compara

- Mesmo **tipo** na mesma **posição** → atualiza e **mantém o state**
- Tipo diferente → desmonta e monta (state perdido)
- Em listas, a posição não chega → a **key** identifica cada item

Consequência

- Um re-render é barato… até haver um filho **pesado**
- Primeiro **medir** (DevTools), depois **compor**
- `memo` / Compiler só no fim — Sessão 4

Keys: a identidade de cada item

A key diz ao React "este é o mesmo de antes" — e com ele, o seu state

1 · index vs id

```
// cada linha tem state interno (uma nota)
{voos.map((v, i) => (
  <LinhaVoo key={i} voo={v} />
))}
// inverter a ordem → a nota fica no voo errado

{voos.map((v) => (
  <LinhaVoo key={v.id} voo={v} />
))}
```

2 · key para reset de state

```
// mudar de voo → formulário limpo,
// sem useEffect nenhum
<NotaDoVoo
  key={selecionado.id}
  voo={selecionado}
/>

// key diferente = componente novo
// → state inicial outra vez
```

`key` estável e única, vinda dos dados (`flight.id`) — **nunca** o índice numa lista que muda. Mudar a key de propósito = reset.



Da teoria à prática

Agora, vamos ver os re-renders

- React DevTools → ⚙ → **Highlight updates when components render**
- Skyline: Select num card — quantos piscam? E a escrever no From?
- Demos `react18-vs-19`: `s2/01` porque renderiza · `s2/03` keys

Sem código novo no Skyline — a solução é o Bloco 03, já a seguir: composição

Bloco 03

Composição

Resolver re-renders e prop drilling com a forma dos componentes — antes de memo

Moving state down

A primeira ferramenta — antes de qualquer memo

antes · state no topo

```
function Pagina() {
  const [count, setCount] = useState(0);
  return (
    <>
      <button onClick={() => setCount((c) => c + 1)}>
        clicks: {count}
      </button>
      <ListaPesada />  {/* renderiza a cada clique */}
    </>
  );
}
```

depois · o state desceu

```
function Contador() {
  const [count, setCount] = useState(0);
  return <button onClick={…}>clicks: {count}</button>;
}

function Pagina() {
  return (
    <>
      <Contador />
      <ListaPesada />  {/* irmã, não filha */}
    </>
  );
}
```

Pôr o state no componente **mais pequeno** que precisa dele. Quem não está por baixo dele deixa de renderizar.

Quando o state não pode descer: children

O componente com state envolve o resto — e recebe-o já feito

antes · renderiza com o painel

```
function Painel() {
  const [aberto, setAberto] = useState(true);
  return (
    <section>
      <button onClick={…}>Recolher</button>
      <ListaPesada />  {/* criada AQUI */}
    </section>
  );
}
```

depois · children

```
function Painel({ children }: { children: ReactNode }) {
  const [aberto, setAberto] = useState(true);
  return (
    <section>
      <button onClick={…}>Recolher</button>
      {children}
    </section>
  );
}
<Painel><ListaPesada /></Painel>  // criada pelo PAI
```

O JSX criado pelo pai é **o mesmo objeto** de antes → o React não o volta a renderizar. Um **slot** é isto mesmo: uma prop `ReactNode` — como o `actions` do `Panel`.



Da teoria ao código

Agora, vamos escrever o código

- `SearchForm`: o filtro **desce** para quem precisa dele
- `FlightSearch` fica só com os voos e o cesto
- `AppShell`: o layout com slots (`brand`, `actions`, `children`)

Ficheiros: features/flights/SearchForm.tsx · FlightSearch.tsx · ui/AppShell.tsx · App.tsx

SearchForm: o filtro desceu

Moving state down, no Skyline

```tsx
export type Filter = { from: string; to: string };

export function SearchForm({ onSearch, onClear }: SearchFormProps) {
  const [filter, setFilter] = useState<Filter>({ from: 'Lisbon', to: 'Porto' });
  const fromRef = useRef<HTMLInputElement>(null);
  const canSearch = filter.from !== '' && filter.to !== '';

  function handleSubmit(event: SubmitEvent<HTMLFormElement>) {
    event.preventDefault();
    onSearch(filter);            // o pai só sabe no submit
  }
  // handleChange e clear(): iguais aos da Sessão 2 (+ onClear())

  return (
    <>
      <title>{`Skyline · ${filter.from || '…'} → ${filter.to || '…'}`}</title>
      <form className="search" onSubmit={handleSubmit}>…</form>
    </>
  );
}
```

Porquê

- O filtro só interessa ao **formulário** → vive aqui
- O pai recebe-o no submit: `onSearch(filter)`
- O `<title>` segue o filtro → veio com ele

FlightSearch: só o state da lista

Os voos encontrados e o cesto — o resto desceu

```tsx
export function FlightSearch() {
  const [flights, setFlights] = useState<Flight[]>([]);
  const [basket, setBasket] = useState<Record<number, boolean>>(…);
  const selectedCount = …;                // igual

  function search(filter: Filter) {
    setFlights(sampleFlights.filter((f) =>
      f.from === filter.from && f.to === filter.to));
  }
  // updateBasket: igual

  return (
    <Panel title="Flight Search" actions={…}>
      <SearchForm onSearch={search} onClear={() => setFlights([])} />
      <FlightList flights={flights} basket={basket}
                  onToggle={updateBasket} />
    </Panel>
  );
}
```

A reter

- State da lista fica em cima
- State do formulário desceu
- Sem `memo`, sem `useCallback`

DevTools → Highlight updates: escrever no From pinta **só** o `SearchForm`. Select num card continua a pintar todos os cards → Sessão 4.

AppShell: um layout com slots

O AppShell decide onde; quem o usa decide o quê

ui/AppShell.tsx

```tsx
type AppShellProps = {
  brand: ReactNode;
  actions?: ReactNode;
  children: ReactNode;
};

export function AppShell({ brand, actions, children }: AppShellProps) {
  return (
    <div className="app">
      <header className="topbar">
        <strong>{brand}</strong>
        {actions}
      </header>
      <main className="content">{children}</main>
    </div>
  );
}
```

App.tsx

```tsx
export default function App() {
  return (
    <AppShell
      brand="Skyline"
      actions={<span>flights, simplified</span>}
    >
      <FlightSearch />
    </AppShell>
  );
}
```

Slots evitam **prop drilling**: o App passa JSX já pronto, em vez de dados que descem três níveis só para chegar lá abaixo.

Resumo: rendering e composição

Hooks

- Snapshot → forma funcional
- Derivado: calcula-se no render
- Hooks no topo, sempre pela mesma ordem

Rendering

- Renderiza: state · pai · context
- Render → reconciliation → commit
- Keys estáveis · key nova = reset

Composição

- Moving state down → `SearchForm`
- `children` / slots: o JSX do pai não volta a renderizar
- `AppShell`: onde vs o quê

Para a Sessão 4: Select num card ainda pinta todos os cards. Quando a composição não chega: `memo`, `useMemo`, `useCallback` — e o React Compiler.

Exercício de revisão

Para casa · no Skyline, a partir da tag sessao-03 · corrigimos no início da Sessão 4

Obrigatório · ≈ 20 min

- `Panel` ganha um slot `footer?: ReactNode`
- No `FlightSearch`: "5 voos encontrados" no footer
- Só aparece quando há voos

Extra

- `FlightList` com um slot `empty`
- Quem usa passa a mensagem: `empty={<p>…</p>}`
- Sem `if` no `FlightSearch`

Bónus

- DevTools → Highlight updates
- Select num card: quem pisca?
- Explicar numa frase porquê

Enunciado no repo: `curso/exercicios/sessao-03.md` · antes de entregar: `npx oxlint` e `npm run build` sem erros

moOngy.
Minds on the move

Obrigado · até quinta, 12 nov

edu@joaogoncalves.net · WhatsApp +351 910 346 437

`github.com/JoaoGoncalves/skyline · tag sessao-03`